Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №7
По дисциплине: “СПП”

Выполнил:
Студент 3 курса
Группы ПО-12
Мартынюк В. В.
Проверила:
Кулик А. Д.

Брест 2026

Цель работы: освоить возможности языка программирования Python в разработке оконных приложений

Задание 1. Построение графических примитивов и надписей

Этот модуль предоставляет класс Cart для управления корзиной товаров,
а также функции для логирования покупок и применения купонов.

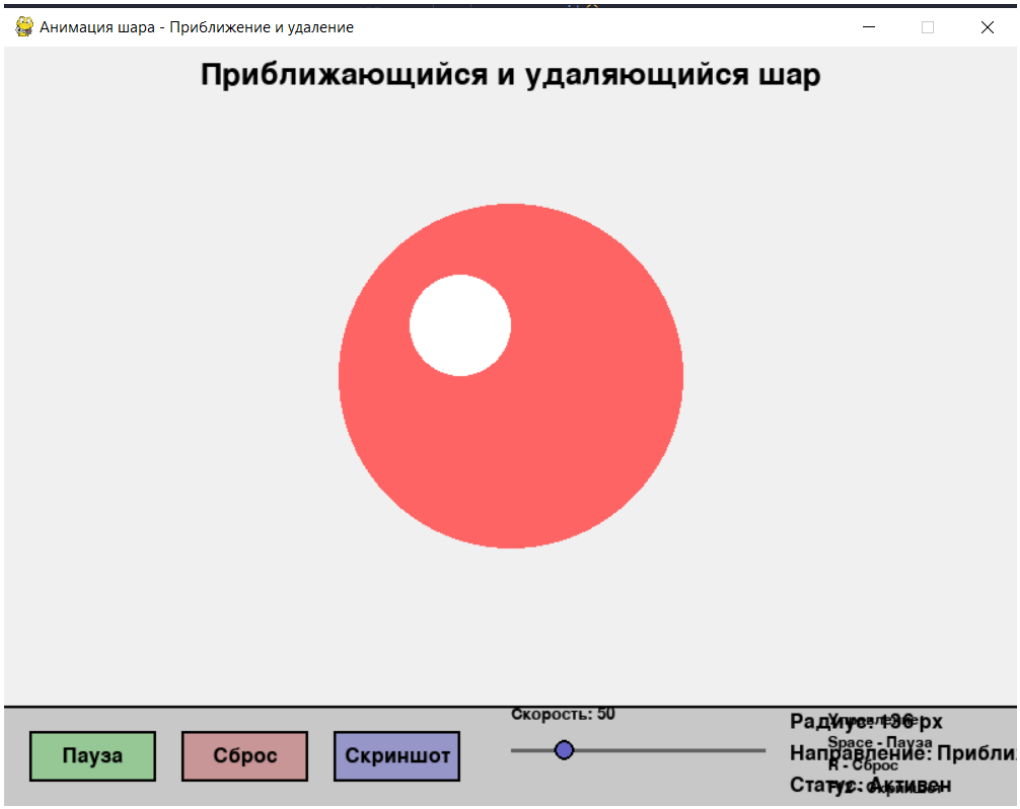
10) Изобразить в форме приближающийся издали и удаляющийся шар. Шар должен двигаться с постоянной скоростью.

```
"""
Главный модуль приложения для анимации шара.
"""

import sys
import os
from datetime import datetime
import pygame
from config import WINDOW_WIDTH, WINDOW_HEIGHT, WINDOW_TITLE, BALL_COLOR,
BALL_START_RADIUS, BACKGROUND_COLOR, FPS
from ball import Ball
from controls import ControlPanel


class BallAnimationApp:
    """Главный класс приложения."""

    def __init__(self):
        """Инициализация приложения."""
        pygame.init()

        self.screen = pygame.display.set_mode((WINDOW_WIDTH, WINDOW_HEIGHT))
        pygame.display.set_caption(WINDOW_TITLE)
        self.clock = pygame.time.Clock()

        # Создаём папку для скриншотов
        self.screenshot_dir = "screenshots"
        if not os.path.exists(self.screenshot_dir):
            os.makedirs(self.screenshot_dir)

        # Создаём шар
        ball_x = WINDOW_WIDTH // 2
        ball_y = (WINDOW_HEIGHT - 80) // 2
        self.ball = Ball(ball_x, ball_y, BALL_START_RADIUS, BALL_COLOR, 50)
```

```
    # Создаём панель управления
    self.control_panel = ControlPanel(WINDOW_WIDTH, WINDOW_HEIGHT)

    # Переменные состояния
    self.running = True
    self.last_time = pygame.time.get_ticks()

def take_screenshot(self) -> None:
    """Сохранение скриншота."""
    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
    filename = f"screenshot_{timestamp}.png"
    filepath = os.path.join(self.screenshot_dir, filename)

    pygame.image.save(self.screen, filepath)
    print(f"Скриншот сохранён: {filepath}")

    # Визуальный 反馈 (мигание экрана)
    flash_surface = pygame.Surface((WINDOW_WIDTH, WINDOW_HEIGHT))
    flash_surface.set_alpha(128)
    flash_surface.fill((255, 255, 255))
    self.screen.blit(flash_surface, (0, 0))
    pygame.display.flip()
    pygame.time.wait(100)

def handle_events(self) -> None:
    """Обработка событий."""
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            self.running = False

        elif event.type == pygame.KEYDOWN:
            if event.key == pygame.K_SPACE:
                self.ball.toggle_pause()
            elif event.key == pygame.K_r:
                self.ball.reset()
            elif event.key == pygame.K_F12:
                self.take_screenshot()
            elif event.key == pygame.K_ESCAPE:
                self.running = False

        # Обработка событий панели управления
        action = self.control_panel.handle_event(event)
        if action == "pause":
            self.ball.toggle_pause()
        elif action == "reset":
            self.ball.reset()
```

```
        elif action == "screenshot":
            self.take_screenshot()

    def update(self, delta_time: float) -> None:
        """Обновление состояния приложения."""
        # Обновляем скорость шара из панели управления
        self.ball.speed = self.control_panel.get_speed()

        # Обновляем шар
        self.ball.update(delta_time)

    def draw(self) -> None:
        """Отрисовка всех элементов."""
        # Очищаем экран
        self.screen.fill(BACKGROUND_COLOR)

        # Рисуем шар
        self.ball.draw(self.screen)

        # Рисуем панель управления
        ball_info = self.ball.get_info()
        self.control_panel.draw(self.screen, ball_info)

        # Добавляем заголовок
        font = pygame.font.Font(None, 36)
        title = font.render("Приближающийся и удаляющийся шар", True, (0, 0, 0))
        self.screen.blit(title, (WINDOW_WIDTH // 2 - title.get_width() // 2, 10))

        # Обновляем экран
        pygame.display.flip()

    def run(self) -> None:
        """Главный цикл приложения."""
        print("""
        ========================================
        Управление:
        Space - Пауза/Продолжить
        R - Сброс
        F12 - Скриншот
        ESC - Выход
        ========================================
        """)

        while self.running:
            # Вычисляем delta time
            current_time = pygame.time.get_ticks()
```

```
        delta_time = min((current_time - self.last_time) / 1000.0, 0.1)
        self.last_time = current_time

        self.handle_events()
        self.update(delta_time)
        self.draw()

        self.clock.tick(FPS)

    pygame.quit()
    sys.exit()


def main():
    """Точка входа в программу."""
    app = BallAnimationApp()
    app.run()


if __name__ == "__main__":
    main()
```



Задание 2. Реализовать построение заданного типа фрактала по варианту 10) Кривая Пеано"""

"""

```
Модуль для построения кривой Пеано (фрактал).
"""

import pygame
import math
from typing import List, Tuple



class PeanoCurve:
    """
    Класс для построения кривой Пеано.

    Кривая Пеано - это пространственно-заполняющая кривая,
    которая проходит через каждую точку квадрата.
    """

    def __init__(self, x: int, y: int, size: int, order: int):
        """
        Инициализация кривой Пеано.

        Args:
            x: координата X верхнего левого угла
            y: координата Y верхнего левого угла
            size: размер области для рисования
            order: порядок кривой (глубина рекурсии)
        """
```

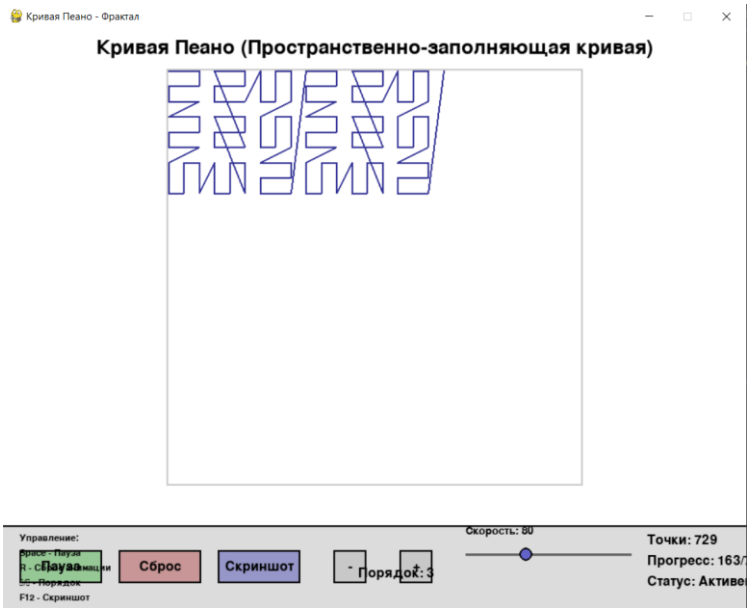
```
    self.x = x
    self.y = y
    self.size = size
    self.order = order
    self.points = []
    self.generate()

def generate(self) -> None:
    """Генерация точек кривой Пеано."""
    self.points = []
    self._peano(self.x, self.y, self.size, self.order, 0)

def _peano(self, x: float, y: float, size: float, order: int, orientation: int) -> None:
    """
    Рекурсивная генерация кривой Пеано.

    Args:
        x, y: координаты текущего блока
        size: размер текущего блока
        order: оставшийся порядок
        orientation: ориентация (0 - горизонтальная, 1 - вертикальная)
    """
    if order == 0:
        self.points.append((x, y))
        return
```

```
    new_size = size / 3

    if orientation == 0:  # Горизонтальная ориентация
        # Рисуем 9 подблоков в порядке Пеано
        self._peano(x, y, new_size, order - 1, 1)
        self._peano(x + new_size, y, new_size, order - 1, 1)
        self._peano(x + 2 * new_size, y, new_size, order - 1, 0)
        self._peano(x + 2 * new_size, y + new_size, new_size, order - 1, 0)
        self._peano(x + new_size, y + new_size, new_size, order - 1, 1)
        self._peano(x, y + new_size, new_size, order - 1, 1)
        self._peano(x, y + 2 * new_size, new_size, order - 1, 0)
        self._peano(x + new_size, y + 2 * new_size, new_size, order - 1, 0)
        self._peano(x + 2 * new_size, y + 2 * new_size, new_size, order - 1, 1)
    else:  # Вертикальная ориентация
        self._peano(x, y, new_size, order - 1, 0)
        self._peano(x, y + new_size, new_size, order - 1, 0)
        self._peano(x, y + 2 * new_size, new_size, order - 1, 1)
        self._peano(x + new_size, y + 2 * new_size, new_size, order - 1, 1)
        self._peano(x + new_size, y + new_size, new_size, order - 1, 0)
        self._peano(x + new_size, y, new_size, order - 1, 0)
        self._peano(x + 2 * new_size, y, new_size, order - 1, 1)
        self._peano(x + 2 * new_size, y + new_size, new_size, order - 1, 1)
        self._peano(x + 2 * new_size, y + 2 * new_size, new_size, order - 1, 0)

def draw(self, screen: pygame.Surface, color: tuple, line_width: int = 2) -> None:
    """
```

```
        Отрисовка кривой Пеано.

        Args:
            screen: поверхность pygame для рисования
            color: цвет линии
            line_width: толщина линии
        """
        if len(self.points) < 2:
            return

        # Рисуем линии между точками
        for i in range(len(self.points) - 1):
            start = (int(self.points[i][0]), int(self.points[i][1]))
            end = (int(self.points[i + 1][0]), int(self.points[i + 1][1]))
            pygame.draw.line(screen, color, start, end, line_width)

    def get_info(self) -> dict:
        """Получение информации о кривой."""
        return {"order": self.order, "points": len(self.points), "size": self.size}


class AnimatedPeanoCurve(PeanoCurve):
    """Анимированная версия кривой Пеано (строится постепенно)."""

    def __init__(self, x: int, y: int, size: int, order: int, animation_speed: float = 50):
        super().__init__(x, y, size, order)
```

```
        self.animation_speed = animation_speed  # точек в секунду
        self.current_point = 0
        self.animation_complete = False
        self.paused = False

    def update(self, delta_time: float) -> None:
        """Обновление анимации."""
        if self.paused or self.animation_complete:
            return

        # Добавляем новые точки в анимацию
        points_to_add = int(self.animation_speed * delta_time)
        self.current_point = min(self.current_point + points_to_add, len(self.points))

        if self.current_point >= len(self.points):
            self.animation_complete = True

    def draw(self, screen: pygame.Surface, color: tuple, line_width: int = 2) -> None:
        """Отрисовка анимированной кривой."""
        if self.current_point < 2:
            return

        # Рисуем только видимую часть кривой
        for i in range(min(self.current_point, len(self.points)) - 1):
            start = (int(self.points[i][0]), int(self.points[i][1]))
            end = (int(self.points[i + 1][0]), int(self.points[i + 1][1]))
```

```
        pygame.draw.line(screen, color, start, end, line_width)


    def reset_animation(self) -> None:
        """Сброс анимации."""
        self.current_point = 0
        self.animation_complete = False
        self.paused = False


    def toggle_pause(self) -> None:
        """Пауза/продолжение анимации."""
        self.paused = not self.paused


    def set_speed(self, speed: float) -> None:
        """Установка скорости анимации."""
        self.animation_speed = max(10, min(200, speed))
```



Вывод: В результате выполнения лабораторной работы были закреплены знания объектно-ориентированного программирования Python при решении задач.